



Lab Sheet 3

Sampling reconstruction and Quantization of DT Signals

Prerequisite

Before starting this section, students should know..

1. A fundamental knowledge on MATLAB
2. A theoretical knowledge on sampling, reconstruction and quantization.

Outcomes

After finishing this lab students should be able to.....

1. understand signal sampling and reconstruction and their implementation using MATLAB.
2. understand quantization and it's implementation using MATLAB.

Outlines of this Lab

1. Lab Session 3.1: Sampling of CT signal
2. Lab Session 3.2: Reconstruction of CT signal
3. Lab Session 3.3: Observation of effect of aliasing
4. Lab Session 3.4: Effect of sampling on sound
5. Lab Session 3.5: Quantization
6. Lab Session 3.6: Effect of quantization on image :
7. Lab Session 3.7: In Lab Evaluation
8. Home work

Lab Session 3.1: Sampling of CT signal

Introduction:

Signals available in our environment are mostly analog and continuous time. As a result we cannot use them for digital signal processing as DSP deals with discrete time signals. As DSP is unable to handle continuous time signals we need to take samples of the signal to make it processable with DSP. Again as the amplitude of the signal is can be of any fractional value. Digital computers are unable to sense all infinitesimal fractional numbers. As a result we need to quantize the discrete samples to process them with DSP.

What is Sampling:

Sampling is a process of converting a continuous time signal into a discrete time signal. As computers are need time to process any data. As a result we cannot process a continuous time signal with a computer. To convert a continuous signal into a discrete signal, samples of the signal are taken periodically.

If $y(t)$ is a continuous time signal, the corresponding discrete signal will be $y(nT)$ where $n=1,2,3,4,\dots$ and T is the sampling interval. We can also write the discrete signal as $y(n)$ with sampling period of T .

Sampling Frequency:

Sampling frequency is the number of samples taken per minute by the continuous to discrete converter. If the sampling period is T , the sampling frequency will be, $f_s = \frac{1}{T}$.

If the sampling frequency is increased the number of samples taken per minute will be increased. Again if the sampling frequency is decreased the number of samples taken per minute will be decreased.

The Nyquist–Shannon sampling theorem:

The Nyquist–Shannon sampling theorem is a fundamental result in the field of information theory, in particular telecommunications and signal processing. The theorem is commonly called Shannon's sampling theorem.

The theorem states that exact reconstruction of a continuous-time baseband signal from its samples is possible if the signal is bandlimited and the sampling frequency is greater than twice the signal bandwidth. The theorem also leads to an effective reconstruction formula.

Sampling using MATLAB

Example 3.1:

Consider an analog signal $x_a(t) = \cos(20\pi t)$, $0 \leq t \leq 1$. It is sampled at $T_s = 0.01, 0.05$ and 0.1 sec intervals to obtain $x(n)$. For each T_s , plot $x(n)$. Comment on your results.

Solution:

MATLAB codes for sampling :

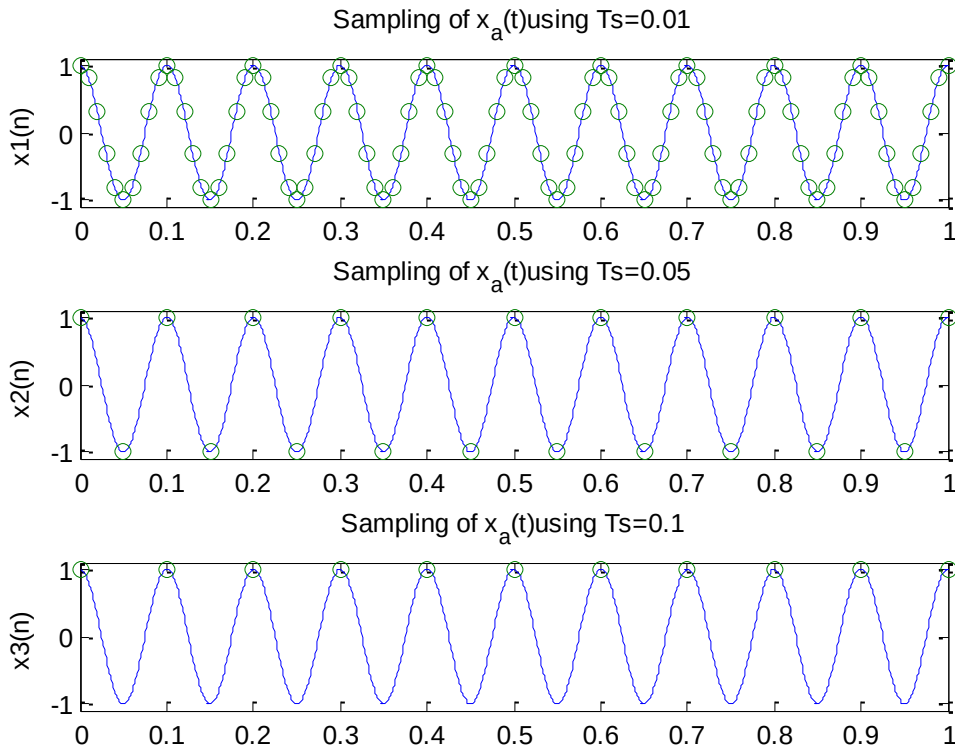
```
t=0:0.001:1;
xa=cos(20*pi*t); % f=10

Ts=0.01; N1=round(1/Ts); n1=0:N1;
x1=cos(20*pi*n1*Ts); subplot(3,1,1);
plot(t,xa,n1*Ts,x1,'o');axis([0,1,-1.1,1.1]);
ylabel('x1(n)'); title('Sampling of x_{a}(t) using Ts=0.01');

Ts=0.05; N2=round(1/Ts); n2=0:N2;
x2=cos(20*pi*n2*Ts); subplot(3,1,2);
plot(t,xa,n2*Ts,x2,'o'); axis([0,1,-1.1,1.1]);
ylabel('x2(n)'); title('Sampling of x_{a}(t) using Ts=0.05');

Ts=0.1; N3=round(1/Ts); n3=0:N3;
x3=cos(20*pi*n3*Ts); subplot(3,1,3);
plot(t,xa,n3*Ts,x3,'o'); axis([0,1,-1.1,1.1]);
ylabel('x3(n)'); title('Sampling of x_{a}(t) using Ts=0.1');
```

Output:



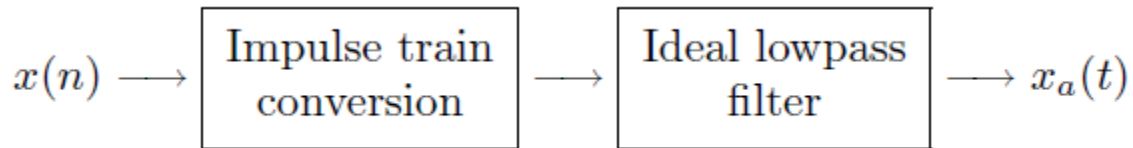
Lab Session 3.2: Reconstruction of CT signal

From the sampling theorem and the preceding examples, it is clear that if we sample band-limited $x_a(t)$ above its Nyquist rate, then we can reconstruct $x_a(t)$ from its samples $x(n)$. This reconstruction can be thought of as a 2-step process:

- (i) First the samples are converted into a weighted impulse train.

$$\sum_{n=-\infty}^{\infty} x(n)\delta(t-nT_s) = \cdots + x(-1)\delta(t+T_s) + x(0)\delta(t) + x(1)\delta(t-T_s) + \cdots$$

- (ii) Then the impulse train is filtered through an ideal analog lowpass filter band-limited to the $[-F_s/2, F_s/2]$ band.



This two-step procedure can be described mathematically using an interpolating formula

$$x_a(t) = \sum_{n=-\infty}^{\infty} x(n) \operatorname{sinc}[F_s(t - nT_s)]$$

Where

$$\operatorname{sinc}(x) = \frac{\sin \pi x}{\pi x}$$

is an interpolating function. We observe that this ideal interpolation is not practically feasible because the entire system is noncausal and hence not realizable.

Example 3.2:

Reconstruct the analog signal $y_a(t)$ from the samples $x(n)$ by means of the sinc interpolation (Use $\Delta t=0.001$.) Estimate the frequency in $y_a(t)$ from your plot. Comment on your results .

Solution:

MATLAB codes

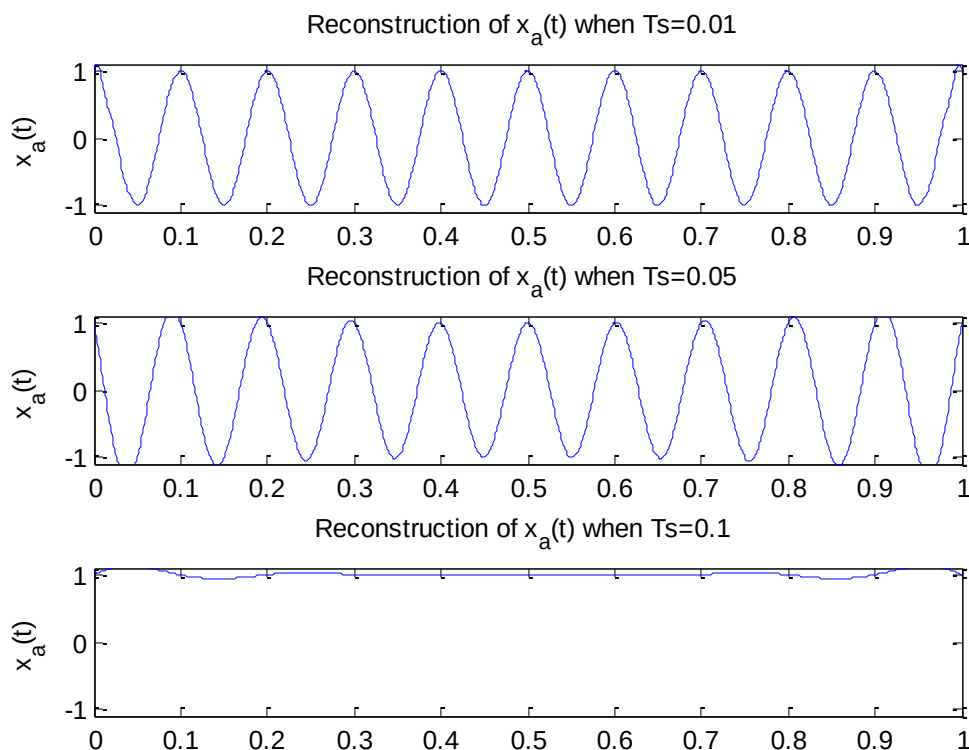
```

Ts=0.01; Fs=1/Ts;
xa1=x1*sinc(Fs*(ones(length(n1),1)*t-(n1*Ts))*ones(1,length(t))));
subplot(3,1,1); plot(t,xa1); axis([0,1,-1.1,1.1]);
ylabel('x_{a}(t)'); title('Reconstruction of x_{a}(t) when Ts=0.01 ');

Ts=0.05; Fs=1/Ts;
xa2=x2*sinc(Fs*(ones(length(n2),1)*t-(n2*Ts))*ones(1,length(t))));
subplot(3,1,2); plot(t,xa2); axis([0,1,-1.1,1.1]);
ylabel('x_{a}(t)'); title('Reconstruction of x_{a}(t) when Ts=0.05 ');

Ts=0.1; Fs=1/Ts;
xa3=x3*sinc(Fs*(ones(length(n3),1)*t-(n3*Ts))*ones(1,length(t))));
subplot(3,1,3); plot(t,xa3); axis([0,1,-1.1,1.1]);
ylabel('x_{a}(t)'); title('Reconstruction of x_{a}(t) when Ts=0.1 ');
  
```

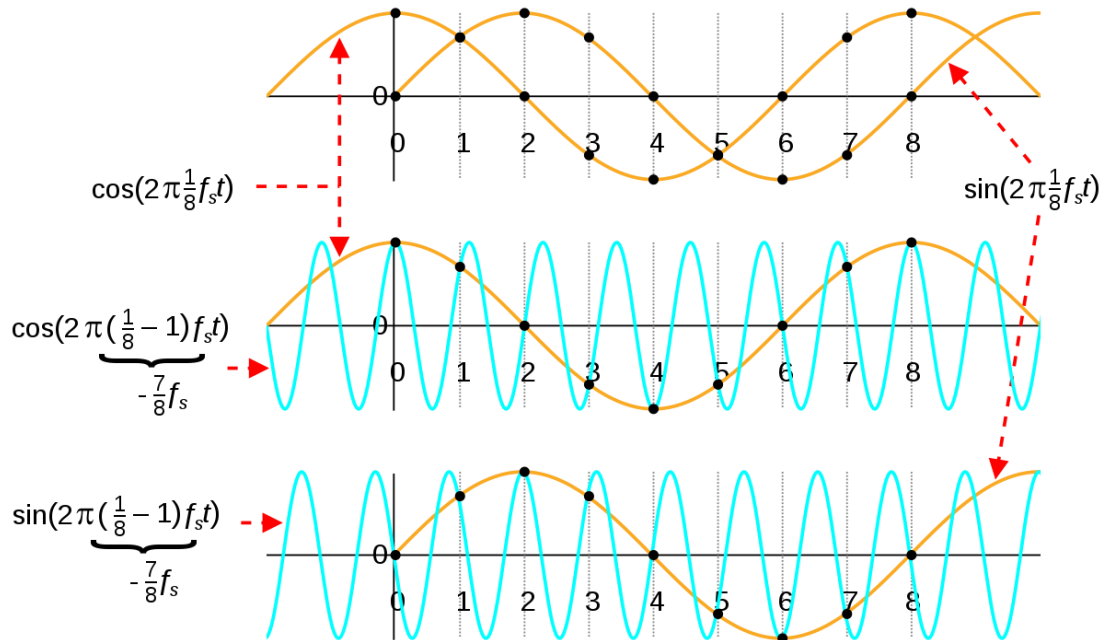
Output:



Comments : From the plots, it is clear that reconstruction from samples at $T_s=0.01$ and 0.05 depicts the original frequency, but the reconstruction for $T_s = 0.1$ shows the original frequency aliased to zero.

Lab Session 3.3: Observation of effect of aliasing:

Consider the following sinusoid signals of two different frequencies. However, they are sampled with a single sampling frequency/interval. In 2nd or 3rd plot, both signals got same sample points and sample values using the same sampling interval. Now the question is: which of two signals will be represented by these samples?



Example 3.3: Plot three functions on same plot as follows :

- (i) $x_1 = \cos 6\pi t$
- (ii) $x_2 = \cos 14\pi t$
- (iii) $x_3 = \cos 26\pi t$ Use $t = nT$, $T = 0.001\text{sec}$ and $n = 500$

Now sample (i) at $T_s = 0.1\text{sec}$. Plot this on the same plot also. What do the results indicate ? Explain.

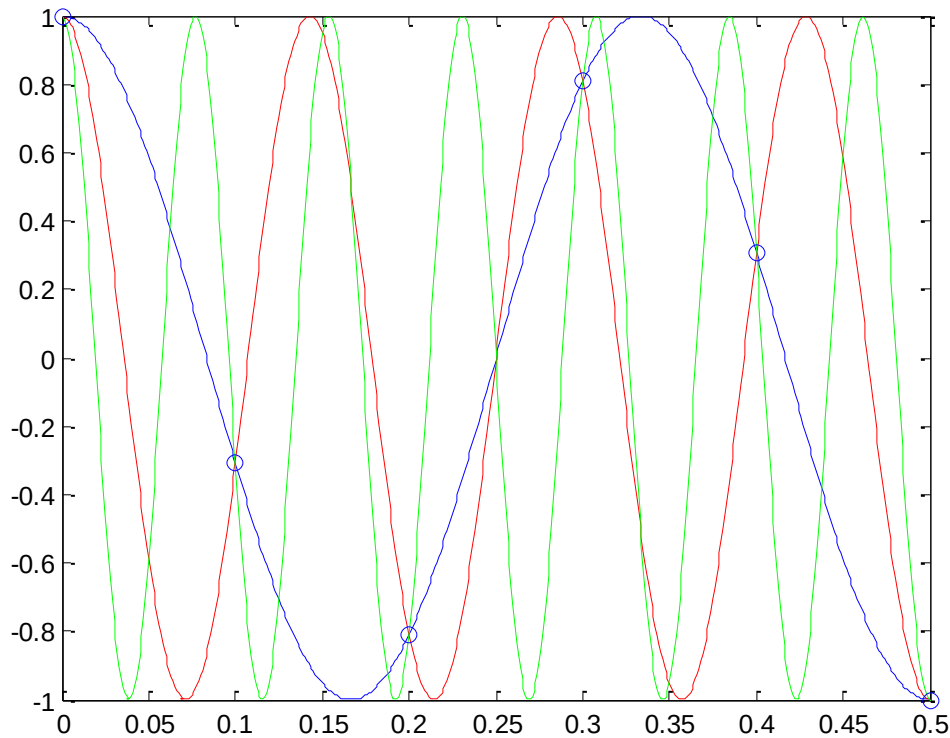
Solution :

MATLAB codes

```
T=0.001; n=0:500;
x1=cos(2*pi*3*n*T); x2=cos(2*pi*7*n*T); x3=cos(2*pi*13*n*T);
plot(n*T,x1); hold on; plot(n*T,x2,'r'); plot(n*T,x3,'g');

Ts=0.1; N=round(1/Ts); n1=0:N/2;
x1=cos(2*pi*3*n1*Ts);
plot(n1*Ts,x1,'o');
```

Output:



Note that, we sampled only x_1 . But, the other two also goes to the circled points . Students must be able to explain this aliasing effect .

Lab Session 3.4:Effect of sampling on sound

3.4.1: USING `sound()` TO DEMONSTRATE THE ALIASING EFFECT

MATLAB has a function `sound()` that can convert a discrete signal sequence into continuous-time sounds. We can use it to demonstrate the effect of aliasing very easily.

Example 3.4:

Introduce the “chirp” phenomena where the instantaneous frequency of some special signal will increase over time. Consider the signal

$$x(t) = \sin\left(\omega_o t + \frac{1}{2}\beta t^2\right)$$

Set $\omega_o = 2\pi(3000)$ rad/sec and $\beta = 2000$ rad/sec².

1. Store in the vector x the samples of the chirp signal on the interval $0 \leq t \leq 1$, and let T is the

same value as above.

2. Use `sound(x, fs)` to play the chirp signal contained in `x`. Can you explain what you just heard?
3. Can you predict the time at which the played signal has zero (or very low) frequency? Use a longer `x` sequence to verify your prediction.

The MATLAB code :

```
f=3000; %frequency of the sine wave
fs=8000; %sampling frequency, try 4k
Ts=1/fs; %sampling time
t= [0:Ts:Ts*20*(fs-1)]; %time samples for 20 sec
x=sin(2*pi*f*t+0.5*2000*t.^2); %sine function for 20 sec
sound(x, fs) %play the wave
```

Task:

- (i) Change `fs=1000, 2000, 3000, 5000, 6000, 8000` respectively and repeat the above code. Can you explain what you just heard in each case? Why the sound is distorted below a certain frequency? Try to detect the sampling frequency below which the sound you have just heard is distorted.

Example 3.5: Playing practical sound:

Steps:

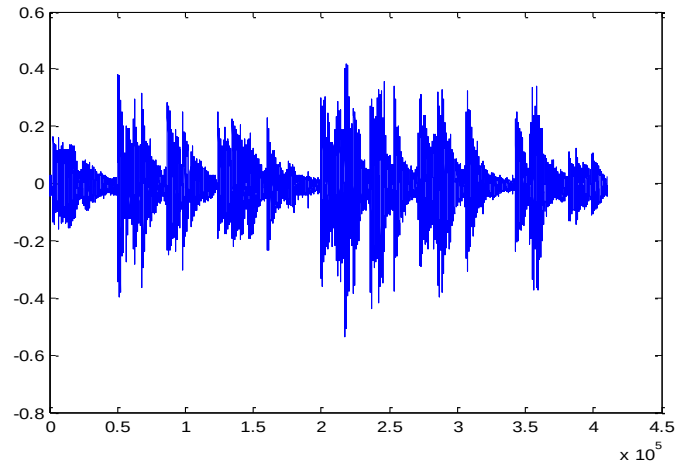
- (i) Load an audio file:

`[y fs]=audioread('audioclip1.wav');` % Place the file 'audioclip1.wav' in the current folder

```
[y fs]=audioread('audioclip1.wav'); % returns the sample rate (fs) in
Hertz and the number of bits per sample (bits) used to encode the data in
the file.
```

- (ii) plot the function : `>> plot(y)`

The following curve will be shown :



(iii) Find the sample rate :

```
>> fs
```

Output: fs =16000

Number of Samples: >> size(y)

Time of Audio sample: >> size(y,1)/fs

(iv) Play the sound :

```
>> soundsc(y,fs)
```

(vi) Find the size of the sound file :

```
>> size(y)
```

Output: ans = 410530 1

(v) Play sound at different sampling frequency:

```
>> soundsc(y,8000)
```

% hear the sound

(vi) Play sound at different sampling frequency:

```
>> soundsc(y,32000)
```

% hear the sound

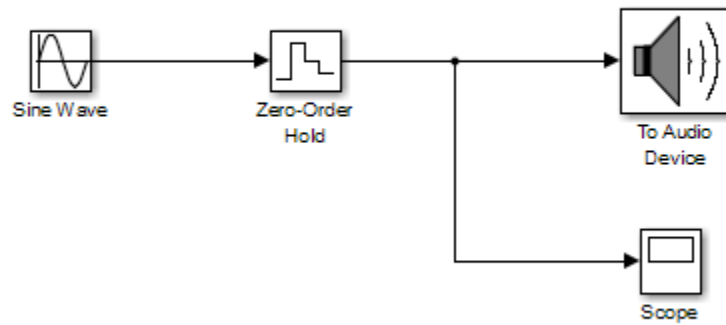
Comment on results of (v) and (vi)

3.4.2: Simulink model TO DEMONSTRATE THE ALIASING EFFECT

Example 3.6:

Step-1:

Construct the following Simulink model.



Step-2:

Choose a frequency of 1kHz ($2\pi \times 1000$ in rad/sec) by double clicking sine wave block.

Choose a sampling times 1/3000, 1/5000, 1/8000, 16000 by double clicking zero order hold block.

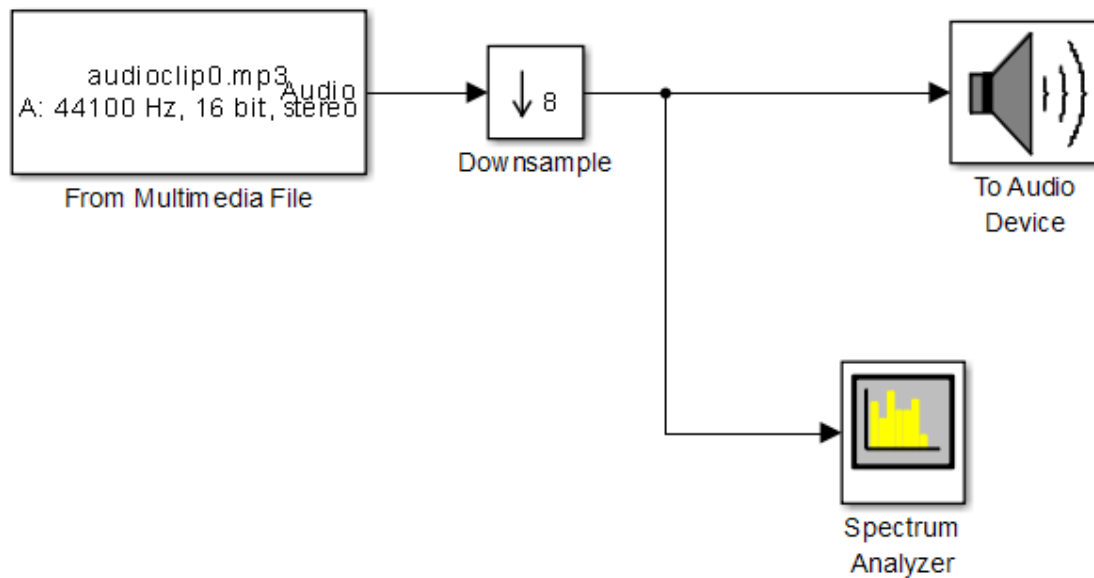
In each case run the program and **you can look at scope** by last 50 samples (Scope > Parameters > History > Limit data points > 50, then right click on figure and select Autoscale)

Observe the sound. In which cases, you hear clear sound? Why?

Example 3.7:

Step-1 :

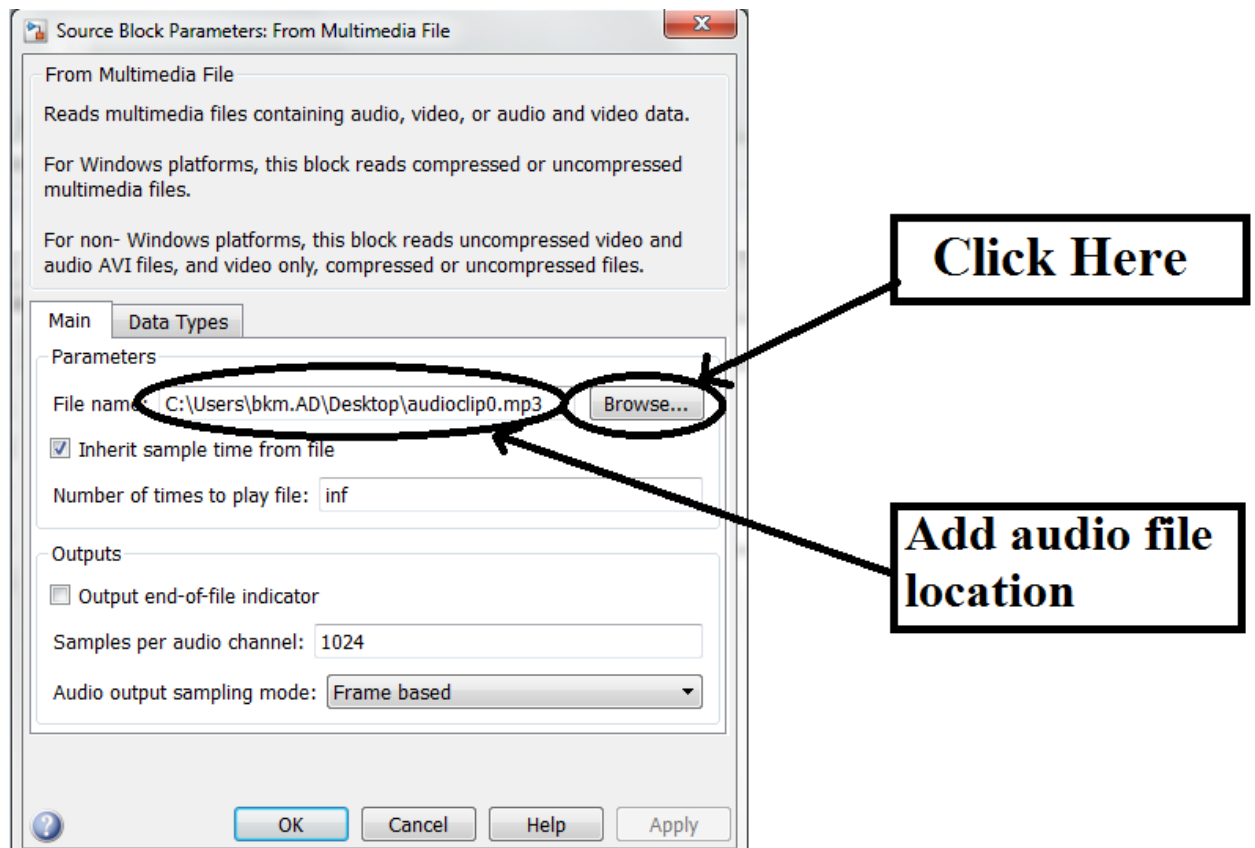
Construct the following SIMULINK model.



Step-2:

Double click the block **From multimedia file**.

Click browse. Add the path of the audio file.



Step-3:

Double click the block **Downsample**. Change the values from **1, 4, 8, 16**. Run program and hear sound in each case.

Comment on overall result.

Lab Session 3.5: Quantization

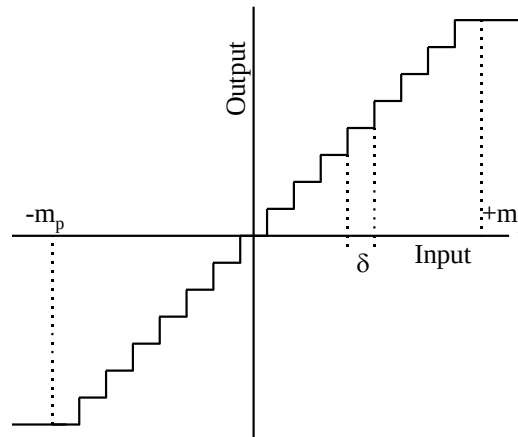
Theoretical background:

In digital signal processing, quantization is the process of approximating a continuous range of values (or a very large set of possible discrete values) by a relatively small set of discrete symbols or integer values. More specifically, a signal can be multi-dimensional and quantization need not be applied to all dimensions. Discrete signals (a common mathematical model) need not be quantized, which can be a point of confusion.

Quantization is the representation of the sampled values of amplitude by a finite set of levels. It converts a continuous-amplitude sample to a discrete-amplitude sample.

In quantization the levels are assigned a binary codeword. All sample values falling between two quantization levels are considered to be located at the centre of the quantization interval. In this manner the quantization process introduces a certain amount of error or distortion into the signal samples. This error known as quantization noise, is minimized by establishing a large number of small quantization intervals. Of course, as the number of quantization intervals increase, so must the

number of bits increase to uniquely identify the quantization intervals. For example, if an analogue voltage level is to be converted to a digital system with 8 discrete levels or quantization steps three bits are required.



We assume that the amplitude of the signal $m(t)$ is confined to the range $(-m_p, +m_p)$. This range $(2m_p)$ is divided into L levels, each of step size δ , given by, $\delta = 2 m_p / L$

A sample amplitude value is approximated by the midpoint of the interval in which it lies. The input/output characteristics of a uniform quantizer is shown in the above figure .

Used Custom function for quantization:

```
function y=uquant(x,N)
%x= original sequence
%N=Number of steps
maxval=max(max(x));
minval=min(min(x));
stepsize=(maxval-minval)/(N-1);
y=round((x-minval)/stepsize)*stepsize+minval;
```

Example 3.8:

Use the function to quantize a vector :

$x = \{1 \ 2 \ -3 \ -4 \ 5 \ 10 \ 12 \ 13 \ 16 \ 11 \ 0 \ -5 \ 7 \ 9 \ 10 \ 16 \}$
 at a level (a) 2 (b) 4 (c) 8 (d) 16

Solution: MATLAB codes

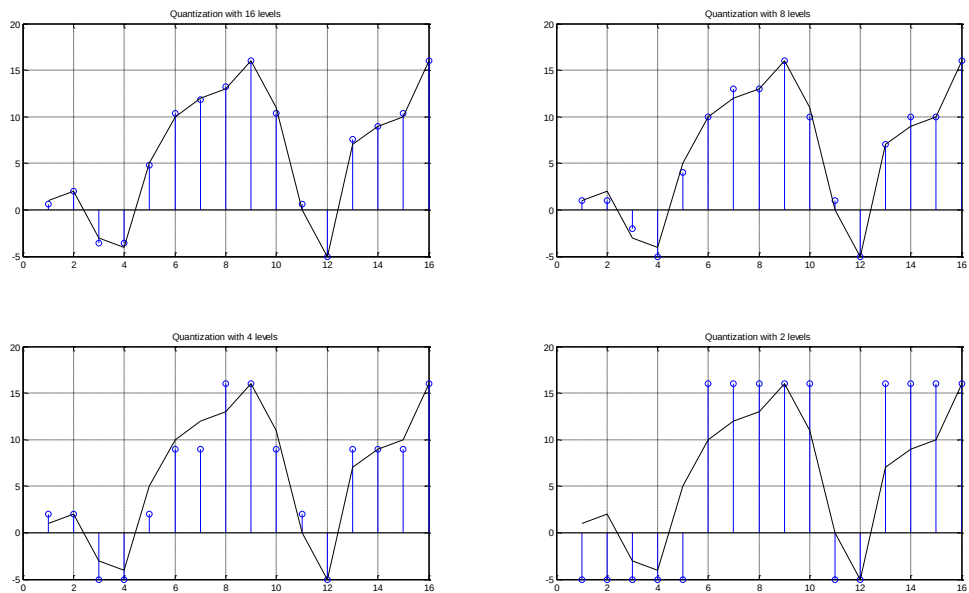
```
x=[1 2 -3 -4 5 10 12 13 16 11 0 -5 7 9 10 16 ];
z=length(x); n=1:z;
y1=uquant(x,16); y2=uquant(x,8);
y3=uquant(x,4); y4=uquant(x,2);
fprintf(' x = %s \n y1 = %s \n y2 = %s \n y3 = %s \n y4 = %s \n ', ...
        num2str(x), num2str(y1), num2str(y2), num2str(y3), num2str(y4) );

figure;
subplot(2,2,1); stem(n,y1); hold on; plot(n,x,'k');
title('Quantization with 16 levels'); grid
subplot(2,2,2); stem(n,y2); hold on; plot(n,x,'k');
title('Quantization with 8 levels'); grid
subplot(2,2,3); stem(n,y3); hold on; plot(n,x,'k');
title('Quantization with 4 levels'); grid
subplot(2,2,4); stem(n,y4); hold on; plot(n,x,'k');
title('Quantization with 2 levels'); grid
```

Output

```
x=1 2 -3 -4 5 10 12 13 16 11 0 -5 7 9 10 16
y1=0.6 2 -3.6 -3.6 4.8 10.4 11.8 13.2 16 10.4
0.6 -5 7.6 9 10.4 16
y2=1 1 -2 -5 4 10 13 13 16 10 1 -5 7 10 10 16
y3=2 2 -5 -5 2 9 9 16 16 9 2 -5 9 9 9 16
y4=-5 -5 -5 -5 -5 16 16 16 16 16 -5 -5 16 16 16 16
```

The figure output is :



Lab Session 3.6: Effect of quantization on image :

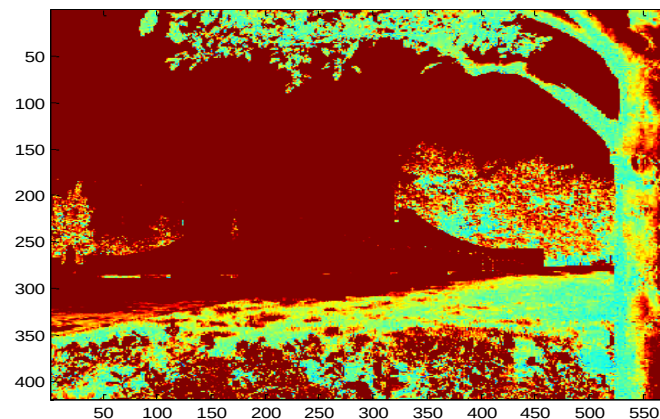
Example 3.9:

Steps

(i) Load an image :

```
x=imread('fountainbw.tif');  
% Make sure that the image is available  
x=double(x);  
image(x)
```

The following image will appear :



(ii) Make the image gray :

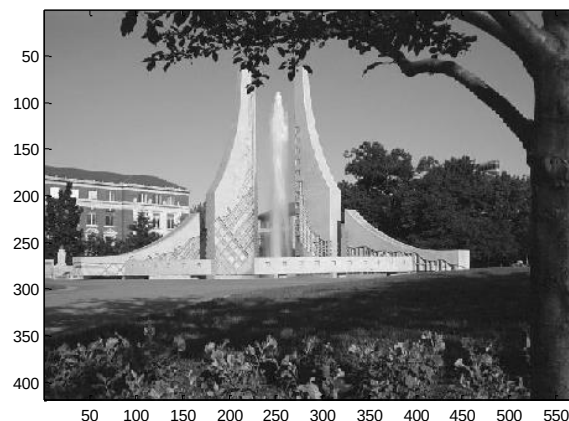
```
>> colormap(gray(256))
```

The figure will be gray colored.

(iii) Make image size square

```
>> axis('image')
```

The image will be like the following:



(iv) Quantize the image at different level and see the effect :

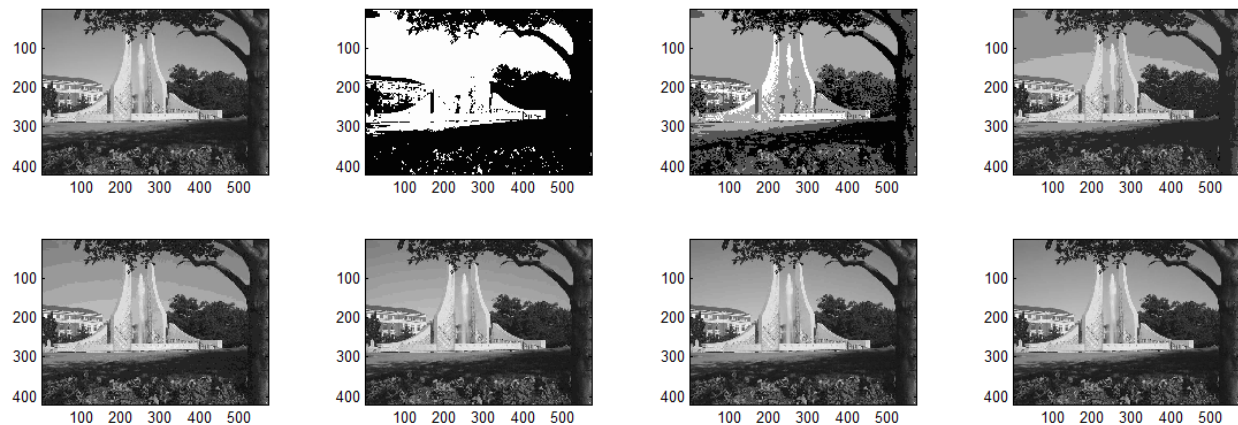
MATLAB code :

```

y=imread('fountainbw.tif');
subplot(2,4,1); image(y); colormap(gray(256));
for b=1:7
    N=2^b; J=uquant(y,N);
    subplot(2,4,b+1); image(J);
colormap(gray(256));
end

```

output :



Lab Session 3.7: In Lab Evaluation

ILE 3.1 : Let, $x_a(t) = e^{-1000|t|}$. Sample the function at $F_s = 5000$ sample/sec and 1000 sample/sec and comment on your result .

Home Work:

HW3.01:

Given a sinusoid waveform with a frequency of 100 Hz,

$$x(t) = 4.5 \sin(2\pi \times 100t)$$

sampled at 8000Hz. Write a MATLAB program to quantize $x(t)$ using 4 bits to obtain and plot the quantized signal, assuming the dynamic range is between -5 to 5 volts.

HW3.02:

An analog signal $x_a(t) = \sin(1000\pi t)$ is sampled using the following sampling intervals . In each case, plot the spectrum in the resulting discrete-time signal .

- $T_s = 0.1\text{ms}$
- $T_s = 1\text{ms}$
- $T_s = 0.01\text{ms}$